

Warm-Up

Design and implement an application that presents a single button to the user. The driver class should be called `TrafficLight`.



The following are the details you should implement:

1. Implement a `TrafficLightButton` class which extends `Button`.
2. The button will start with background color `lawngreen` and each time it is clicked, it will cycle through yellow then red then back to `lawngreen`.
3. Define a `Color` array constant named `COLORS` and use a list initializer to fill the array with `lawngreen`, `yellow` and `red` (in that order); utilize the `Color` enumeration provided.
4. Keep track of the current color using an index into an array of possible colors named `colorIndex`.
5. Set the preferred size of your button to 200 x 200.
6. Set the initial color to `lawngreen`.
7. Don't forget to add an event handler to your button's `setOnAction()` method. Your event handler method should increment the `colorIndex` and change the background color of the button accordingly. Note- when the index gets to the end of the color array, it should loop back to 0 again.

Lab Details

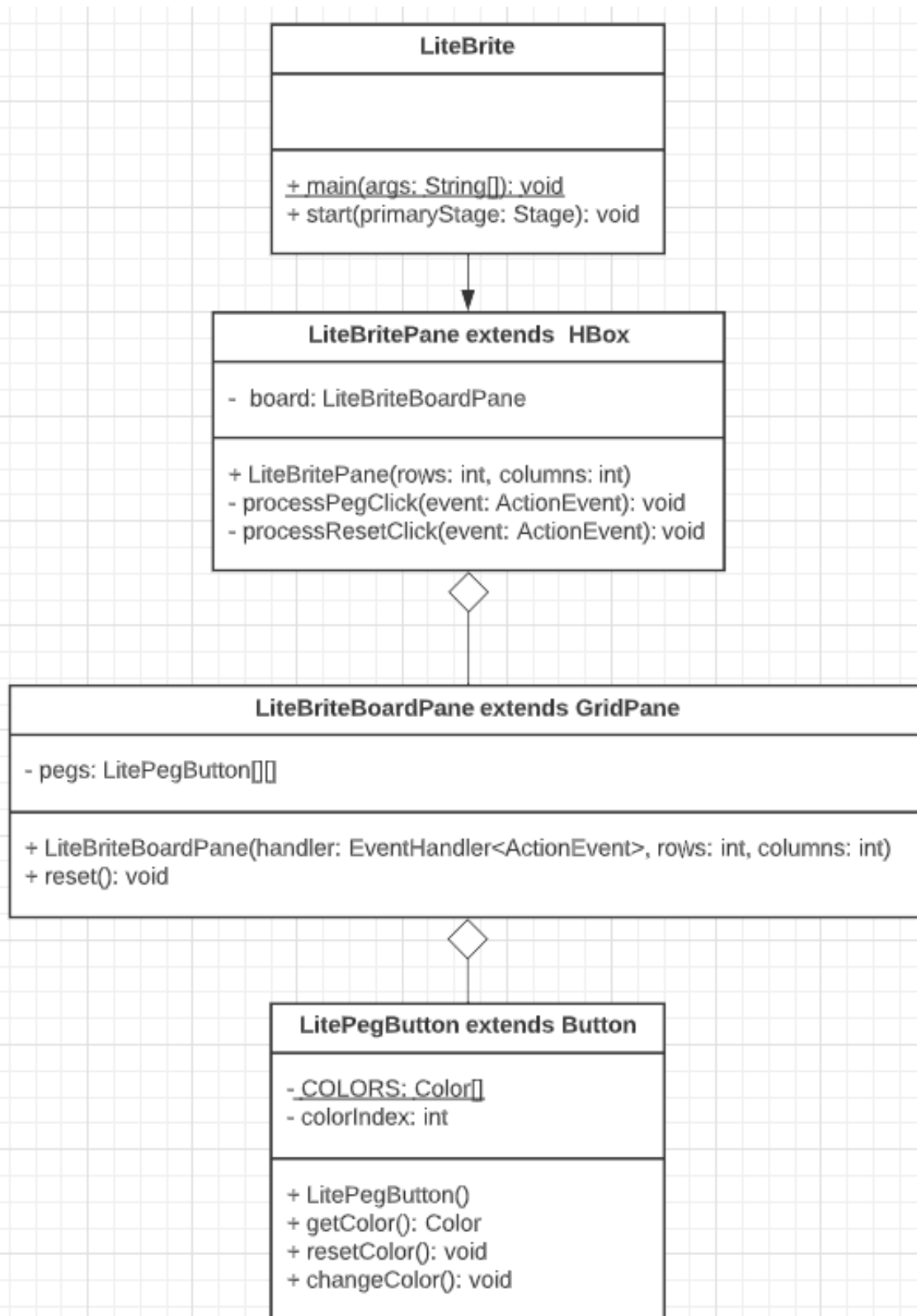
Overview

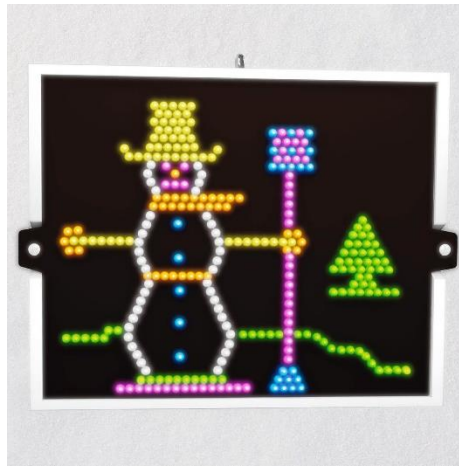
In this lab, you will create a GUI modeled after Lite-Brite.



Objectives

1. Write your first Graphical User Interface (GUI) in Java.
2. Design and write classes to model your GUI data.
3. Manage the GUI layout using sub-panels.
4. Use a `GridPane` for displaying a 2D array of objects.
5. Use `Button`, `HBox` and `GridPane`.
6. Implement and add event handlers to GUI elements to handle user interaction events.





Getting Started

1. Create a Lab 09 project.
2. Copy/paste into the folder `Color.java`, `LiteBrite.java` and `LiteBritePane.java`
3. You will use the existing `LiteBrite` driver class and `LiteBritePane` class. The `LiteBritePane` class will not compile until you implement your other classes.
4. You will write your own `LiteBriteBoardPane` and `LitePegButton` classes.
5. You will be using your knowledge in coding the `TrafficLight` application here.

Specifications

LiteBrite and LiteBritePane classes:

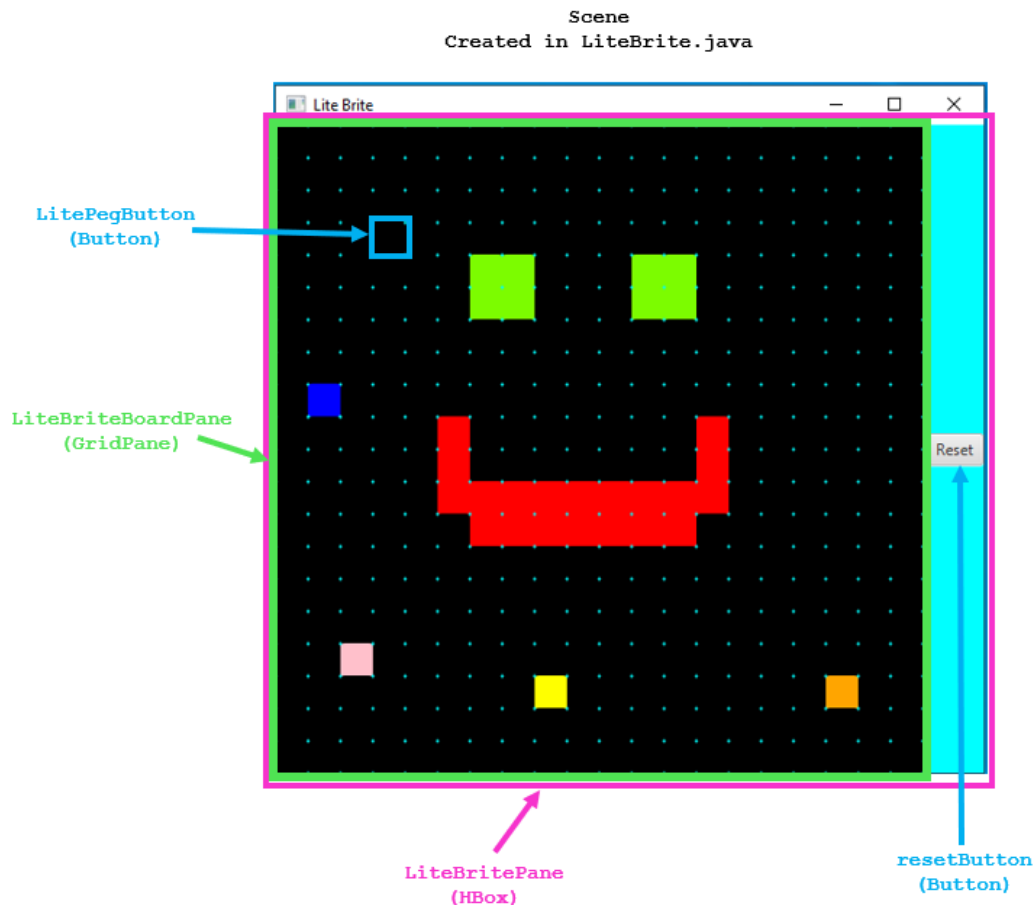
The `LiteBrite` driver class is already complete. It creates a new `Scene` and adds a `LiteBritePane` to the scene.

The `LiteBritePane` class is mostly complete, you will just need to add a second event handler to handle `LitePegButton` clicks. It extends `HBox`, which means it *IS* an `HBox`. It will not compile until you implement your `LiteBriteBoardPane` and `LitePegButton` classes.

LiteBriteBoardPane and LitePegButton classes:

1. Create classes for `LitePegButton` and `LiteBriteBoardPane`.
2. Use the UML diagram below to guide you with the methods and private variables required.
3. `LitePegButton` must extend `Button`. It will represent the state of a single peg button. Implement the methods as shown in the class design image below. This class will be similar in design to the `TrafficLightButton` class but with a size of 25x25 and a color array consisting of black, purple, blue, lawngreen, yellow, orange, red, pink, and white (in that order). The initial background color should be black.
4. `LiteBriteBoardPane` must extend `GridPane`. It will represent the state of the entire board. Implement the methods as shown in the class design below. Remember that you will be managing and adding all the `LitePegButton` objects to this pane using the `GridPane`'s `add` method.

Here are a few images to help you visualize the classes you will be implementing and the GUI components you will be adding.



Rubric

- ❖ Do the programs work as described? (Needs my sign-off): **10 points**
- ❖ Did you follow the directions along with proper style and coding practices: **10 points**
- ❖ **Total: 20 points**

Submitting the Lab

Compress the submission files (see below) into a .zip file named: **Lab09_LastName_FirstName.zip** When you are done and ready to submit, submit the following files in the .zip file:

- TrafficLight.java
- TrafficLightButton.java
- LiteBrite.java
- LiteBritePane.java
- LiteBriteBoardPane.java
- LitePegButton.java
- Color.java